Lab 5 – Prolog Fundamentals

Group #: 3

Joey Issa - 300219818

Garrett Winslow - 300149981

Yannick Vaillancourt – 300226628

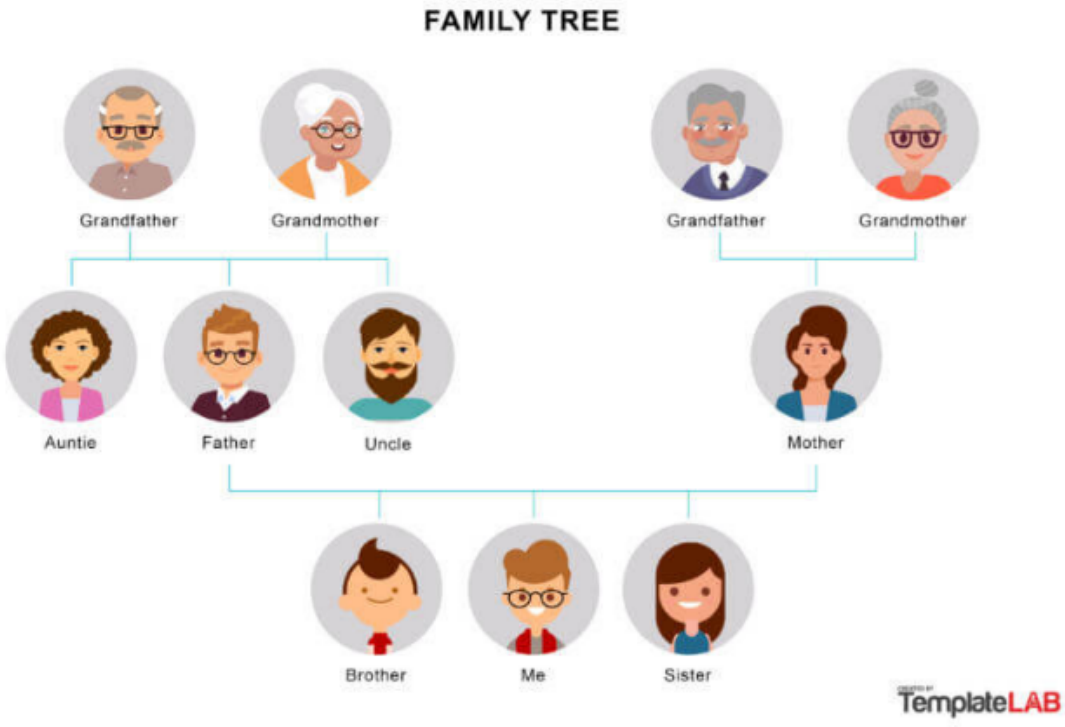
Task A report

Explanation of Facts and Rules:

1. Family Facts: Describe how you defined the family tree using `parent/2`, `male/1`, and `female/1` facts. Explain why these facts are essential for representing relationships and gender in the family tree.

Defining family trees with parent(john, mary), male(fred), and female(mary) facts allows us to establish some basic fundamental relationships between the individuals. If we know that male(fred) and female(mary) we can determine multiple things, like whether they are a grandmother or grandfather, or sister or bother, for this lab the facts of male and female were not used or inherently needed to determine ancestor or grandparent, but it would be very useful if i wanted to get into the specifics.

Family trees are based off of people's lineage of different parent groups:



Rules for Relationships:

1. Explain each of the following predicates and how they were implemented:
    - `sibling/2`: Describe the logic used to identify siblings and how you ensured that the two individuals are not the same person.
    - `grandparent/2`: Explain how you used the `parent/2` predicate to chain together parent relationships and identify grandparents.
    - `ancestor/2`: Describe the recursive logic used to identify an ancestor and how recursion allows for tracing the lineage through multiple generations.

```
/* Question 1 */
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
```

For siblings, we must ensure that they have a same parent (Z) which we do with the logic
parent(Z, X), parent(Z, X)
This states that person X and Y must have a common parent Z. In order to ensure that they are not the same person, we end the logic with the comparator **X \= Y.** This excludes the case where person X and Y are the same

```
/* Question 2 */
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

For grandparents, we check there is every a case that two people X (the first parameter we think is the grandparent of) and Y exist where X is the parent of Z, and Z is a parent of some other individual Y.

This means that X is a grandparent to Y, where X is the parent of Z and Z is the parent of Y.

```
/* Question 3 */
ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

For ancestors, we see if some person X is related (in the past tense) to some person Y. We check there exists the case that person X 'fathered' Y recursively going through every individual case.

We check the first case **ancestor(X, Y) :- parent(X, Y).** which is the base case, if this is true it ends the recursive loop

Then we encounter the recursive case: **ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).** Which checks if there is some Z where X is the parent of Z and Z is an ancestor of Y. This creates a recursive call stack filled with trying to narrow down every R which is the parent of Q till we get to the case that the starting individuals X and Y are related.

Testing and Output Validation:

```
?- male(tom).
true.
```

Met expected outputs.

```
?- parent(mary, Child).
Child = ann ;
Child = fred.
```

Met expected outputs.

```
?-  sibling(ann, Sibling).
Sibling = fred.
```

Met expected outputs.

```
?- grandparent(GP, fred).
GP = john .
```

Met expected outputs.

```
?- parent(john, Child), female(Child).
Child = mary ;
false.
```

Met expected outputs.

```
?- ancestor(Anc, fred).
Anc = mary ;
Anc = john ;
false.
```

Met expected outputs.

```
?- grandparents(john, GC), female(GC).
Correct to: "grandparent(john,GC)"? yes
GC = ann ;
GC = liz.
```

Met expected outputs.

Task B report

1. The base case just says "when the list is empty stop recursing", without it prolog will continue to waste everyone's time taking the sum of an empty list.

2. When head is an odd number then the first section fails (0 is NOT Head mod 2). That means we branch to the next possible true (the other side of the or) which takes the sum counting the head.

3. If the number is even then we don't branch and we simply recurs, not counting the head.

```
Welcome to SWI-Prolog (threaded, 64 bits, version 8.4.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- sum_odd_numbers([1, 2, 3, 4, 5], Sum).
Sum = 9.

?- sum_odd_numbers([0, 7, 3, 8], Sum).
Sum = 10.

?- sum_odd_numbers([], Sum).
Sum = 0.

?-
```

Task C report

Permutation:

I used the built-in ‘permutation/2’ predicate. It has two arguments: the first is a list of all 4 house colors, representing what items the lists should generate permutations of, and the destination list that these permutations get stored as, providing the ‘Houses’ list for any permutation that succeeds.

Helper predicates:

‘next_to/3’ checks if the first 2 arguments are found back-to-back in the list, while ‘not_next_to/3’ checks for this to not be the case. The base cases for next_to are: ‘we’ve reached the end of the list and have no more elements’, indicating they were not found back-to-back (in the order 1st element -> 2nd element), and ‘1st element is the head of the list, and the 2nd element is the head of the tail of the list’, meaning they were found, with the recursive step simply trying again with the tail of the list. ‘not_next_to/3’ simply checks if the negation of next_to is true (returning the opposite of next_to).

Valid solutions:

```
?- solve_puzzle(Houses,GreenIndex).
Houses = ['Yellow', 'Red', 'Blue', 'Green'],
GreenIndex = 4 ;
Houses = ['Green', 'Red', 'Blue', 'Yellow'],
GreenIndex = 1 ;
```

These solutions are valid because all clues have been met. Red is always to the left of Blue (taking advantage of the fact that ‘next_to’ technically only checks if the sequence [...,X,Y,...] exists in the list and not the other way around). The second clue effectively finds the index of ‘Green’ since the treasure is always in the green house, and thus will never have a mismatch. The third clue ensured ‘Yellow’ and ‘Green’ were never side by side (in either direction, thus requiring both ‘not_next_to’s). Lastly the fourth clue ensures the ‘Green’ house is not second from the left, but all other outcomes are acceptable.

The fourth clue is actually unneeded, because if ‘Red’ is always next to ‘Blue’, but ‘Green’ cannot be next to ‘Yellow’, then they must be split, with ‘Red’ & ‘Blue’ taking up the middle 2 spots. This therefore prevents ‘Green’ from being the 2nd house entirely, and the remaining permutations simply require getting the index of the ‘Green’ house to obtain the treasure.